

Machine Learning Report

Decision Tree Classifier

兰天丞 1120230272

Apr 22nd, 2025

Abstract

This report details the implementation and evaluation of a Decision Tree classifier for binary classification tasks. The goal was to build an end-to-end learning system from scratch, focusing on entropy-based splits using mutual information. The experiments were conducted on four datasets: ‘politician’, ‘education’, ‘small’, and ‘mushrooms’, each representing distinct classification challenges.

Keywords: Decision Tree, Classifier, Machine Learning, Entropy, Mutual Information

Contents

1	Introduction	5
1.1	Overview	5
1.2	Database description	5
2	Program #1: Inspecting the Data	6
2.1	requirements analysis	6
2.2	Definition	6
2.2.1	Entropy	6
2.2.2	Error Rate	7
2.2.3	majority vote	7
2.3	Implementation	8
2.3.1	Key function	8
2.3.2	Program process	9
2.4	Test	10
2.5	Setup and Use	11
2.6	Final Results	11
3	Program #2: Decision Tree Learner	12
3.1	Requirements Analysis	12
3.2	Definition	12
3.2.1	Decision Tree	12
3.2.2	Mutual Information	14
3.3	Implementation	14
3.3.1	Helper function	15
3.3.2	TreeNode class	16
3.3.3	Splitting Mechanism	18
3.3.4	Build Decision Tree	21
3.3.5	Task function	21
3.3.6	Program Process	23

3.4	Test	26
3.4.1	Test 1	26
3.4.2	Test 2	27
3.5	Perfomance Analysis	28
3.5.1	Relation between max_depth and error rate	28
3.5.2	Futher Work	30
4	Disucuss and Conclusion	30
4.1	Python3? Python? Python!	30
4.2	Conclusion	31
5	AI Using Report	31

1 Introduction

1.1 Overview

The goal in this assignment is to implement a binary classifier, entirely from scratch—specifically a Decision Tree learner. I finished the three tasks by running some end-to-end experiments. Finally, I wrote my programs in Python, then used the command line tool to run the program, which is quite different from my previous Python project

1.2 Database description

In this experiments, the database is quite clean and organized, which makes the experiments easier to implement. Each dataset contains attributes and labels and is already split into training and testing data. The first line of each .tsv file contains the name of each attribute, and the class is always the last column. So we don't need to do the data cleaning tasks, we can simply use some python operations to fully use the data.

However, the data is in tsv format. TSV (Tab-Separated Values) files are a simple and practical format for storing and representing various types of data, including text and numerical values. Each field in a TSV file is separated by a tab character, making it easy to read and write data in a tabular format. But I didn't use this format before, so I use **ChatGPT**¹ to help me know how to read the data.

¹In this report, the highlights parts mean I use AI tool here. There is also an AI using report in the end.

2 Program #1: Inspecting the Data

2.1 requirements analysis

Write a program `inspection.py|java|cpp` to calculate the label entropy at the root (i.e. the entropy of the labels before any splits) and the error rate (the percent of incorrectly classified instances) of classifying using a majority vote (picking the label with the most examples). You do not need to look at the values of any of the attributes to do these calculations, knowing the labels of each example is sufficient. Entropy should be calculated in bits using log base 2.

In addition, we should use the command line to run the program. Here is the command line I used:

```
1 python inspecting.py <input> <output>
```

It means that the program will get 2 parameters, the first one is the input file name, the other one is the output file name. Fianlly, it will write the following information to the output file:

```
entropy: <entropy value>
error rate: <error rate value>
```

2.2 Definition

Here we give some definitions about the used terms in this program.

2.2.1 Entropy

Entropy, denoted as $H(Y)$, is a fundamental concept in information theory that measures the uncertainty or impurity in a set of labels. For a discrete random variable Y with possible values $y_1, y_2, \dots, y_n$ and corresponding probabilities $P(y_1), P(y_2), \dots, P(y_n)$, the entropy is defined as:

$$H(Y) = - \sum_{i=1}^n P(y_i) \log_2 P(y_i)$$

In the context of decision trees, entropy is used to quantify the disorder in a dataset and to guide the selection of the best feature to split on. A lower entropy indicates a more pure (less uncertain) set, while a higher entropy indicates more disorder.

For example, if a dataset has two classes with equal probability, the entropy reaches its maximum of 1 bit. Conversely, if all instances belong to the same class, the entropy is 0.

2.2.2 Error Rate

Error rate is a simple and intuitive metric used to evaluate the performance of a classification model. It is defined as the proportion of incorrectly classified instances among the total number of instances. Formally, if a dataset contains N samples, and the model misclassifies E of them, then the error rate is given by:

$$\text{Error Rate} = \frac{E}{N}$$

In other words, error rate measures the fraction of predictions that do not match the true labels. A lower error rate indicates better model performance, while a higher error rate suggests poor classification accuracy. Error rate is complementary to accuracy, where:

$$\text{Accuracy} = 1 - \text{Error Rate}$$

This metric is widely used in machine learning and pattern recognition tasks to assess how well a model generalizes to unseen data.

2.2.3 majority vote

Majority vote is a simple and commonly used method in classification tasks to decide the predicted class of a sample when multiple predictions are available. The principle is to choose the class that receives the highest number of votes among all candidates.

Formally, given a set of predicted labels $\{y_1, y_2, \dots, y_m\}$ from m classifiers or neighbors, the majority vote selects the label y^* such that:

$$y^* = \arg \max_y \sum_{i=1}^m \mathbf{1}(y_i = y)$$

where $\mathbf{1}(\cdot)$ is the indicator function that equals 1 if the condition is true, and 0 otherwise.

This approach is widely used in ensemble learning methods, such as Random Forests, and in k-Nearest Neighbors (k-NN) classifiers to aggregate multiple decisions into a single final prediction. Majority vote helps to reduce the variance and improve the robustness of the classification results.

2.3 Implementation

Here we give some implementation details about the program.

2.3.1 Key function

The main function of this program is to calculate the entropy and error rate of the input file. Based on the formula given above, we can simply achieve this function in Python.

```
1 def compute_entropy(labels):
2     #here labels is an array to store all the label appear in
       ↪ the file
3     total = len(labels)
4     labels_count = Counter(labels)
5     #use the counter to find the frequency of each labels
6     entropy = 0.0
7
8     for i in labels_count.values():
9         p = i/total
10        entropy -= p*math.log2(p)
11
12    return entropy
13
```



```
14 def compute_error(labels):
15     label_counts = Counter(labels)
16     most_common_label, most_common_count = label_counts.
        ↪ most_common(1)[0]
17
18     # in the majority vote
19     error_count = len(labels) - most_common_count
20     error_rate = error_count / len(labels)
21     return error_rate
```

2.3.2 Program process

Based on the thought of process-oriented programming, we can divide the program into several steps:

- Read the input file and get the labels from the file.
- Calculate the entropy and error rate of the labels.
- Write the results to the output file.
- Use the command line to run the program.

We can use the file operations to do the first and final step, and then we can use the function we defined above to do the second step.

And we import the sys modules to deal with the command line parameters, which helps us to interact with the user. If the user's input is illegal, we can use the `sys.exit(1)` to stop the program, and print out "Usage: python inspection.py <input> <output>" to tell the user how to use the program.

The modules we need to import are as follows:

```
1 import sys
2 import math
3 import csv
4 from collections import Counter
```

Then we can use the following `code` to read the input file and get the labels from the file.

```
1 def main():
2     if len(sys.argv) != 3:
3         #user's inputs are illegal
4         print("Usage: python inspection.py <input> <output>")
5         sys.exit(1) # stop
6     # sys.argv actually is an array to store the user's input
7     ↪ I think
8     input_file = sys.argv[1]
9     output_file = sys.argv[2]
10
11    # open the file
12    with open(input_file, 'r') as f:
13        labels = []
14        reader = csv.reader(f, delimiter='\\t') # read the
15        ↪ tsv file, \\t is the divider
16        header = next(reader) # skip the first row cause
17        ↪ this is the tittle
18        label_index = -1 # labels are in the last column
19        for row in reader:
20            labels.append(row[label_index])
21
22    #compute
23    entropy = compute_entropy(labels)
24    error_rate = compute_error(labels)
25
26    with open(output_file, 'w') as fo:
27        fo.write(f'entropy: {entropy!r}\\n')
28        fo.write(f'error: {error_rate!r}\\n')
```

Finally, use the program interface to run the program.

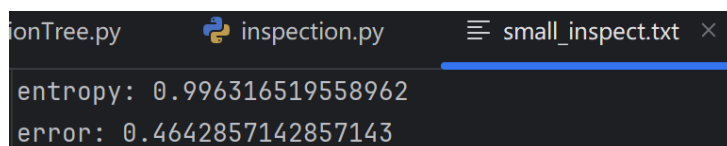
```
1 if __name__ == "__main__":
2     main()
```

2.4 Test

Finally, we use the following command to test the program:

```
1 python inspection.py small_train.tsv small_inspect.text
```

After running, we can get a txt file in the same direction. Here is the content of the file, which is as same as the given answer:



```

entropy: 0.996316519558962
error: 0.4642857142857143

```

Figure 1: The content of small_inspect.txt

2.5 Setup and Use

We can then use the command line to run the following command, and we can get the result of all the data we have:

```

1 python inspection.py politicians_train.tsv
   ↪ politicians_inspect.txt
2 python inspection.py education_train.tsv education_inspect.
   ↪ txt
3 python inspection.py mushrooms_train.tsv mushrooms_inspect.
   ↪ txt

```

2.6 Final Results

Here is the entropy and error rate of the four datasets:

Filename	small_train	politicians_train
Entropy	0.996316519558962	0.9905894286537543
Error rate	0.4642857142857143	0.4429530201342282
Filename	education_train	mushrooms_train
Entropy	0.9097361225311662	0.9626147059982517
Error rate	0.32499999999999996	0.38666666666666666

Table 1: Entropy and Error Rate for Different Training Files

3 Program #2: Decision Tree Learner

3.1 Requirements Analysis

In `decisionTree.py`, we implement a Decision Tree learner. This file learned a decision tree with a specified maximum depth, print the decision tree in a specified format, predict the labels of the training and testing examples, and calculate training and testing errors.

Here is the detailed requirements:

- Use mutual information to determine which attribute to split on.
- As a stopping rule, only split on an attribute if the mutual information is > 0 .
- You should split with replacement. That is, when you split on a column, you should retain this column in child datasets.
- Do not grow the tree beyond a max-depth specified on the command line.
- Use a majority vote of the labels at each leaf to make classification decisions. If the vote is tied, choose the label that comes last in the lexicographical order.
- It is possible for attributes to have equal values for mutual information. In this case, you should split on the first attribute to break ties.

3.2 Definition

Here is the basic definition of the terms we used in this program:

3.2.1 Decision Tree

A decision tree is a tree-structured classifier that recursively partitions the input space based on attribute values to make predictions. It is commonly

used in classification and regression tasks due to its simplicity, interpretability, and effectiveness.

The construction of a decision tree follows a greedy, top-down recursive approach, often referred to as the **ID3** or **C4.5** algorithm (depending on the criterion used). At each node, the algorithm selects the best attribute to split the data, usually based on *information gain* or *Gini impurity*. The recursion continues until a stopping condition is met (e.g., pure node, max depth, or no remaining attributes).

Algorithm 1 DecisionTreeTrain(Data, Attributes)

1: **Input:**

Data: $D = (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Attributes: A list of attributes (features) to split on.

2: **Output:**

A decision tree that classifies the input data

3: **if** all examples in Data belong to the same class **then**

4: **return** a leaf node with that class label

5: **end if**

6: **if** Attributes is empty or Data is too small **then**

7: **return** a leaf node with the majority class in Data

8: **end if**

9: Select the best attribute A using a criterion (e.g., mutual information)

10: Create a decision node that splits on A

11: **for** each value v of attribute A **do**

12: Let D_v be the subset of Data where $A = v$

13: Recursively call **DecisionTreeTrain**(D_v , Attributes $\setminus \{A\}$)

14: Attach the resulting subtree to the decision node

15: **end for**

16: **return** the decision node

This recursive procedure constructs the tree until it is fully grown or meets stopping criteria such as maximum depth or minimum samples per

node. To avoid overfitting, pruning techniques may be applied after training.

3.2.2 Mutual Information

Mutual Information (MI) is a key metric used in decision tree learning to determine the optimal attribute for splitting the data. It measures the amount of information gained about the target variable Y by knowing the value of an attribute X .

In the context of binary classification and binary attributes, the mutual information between a label Y and an attribute X is computed as:

$$I(Y; X) = H(Y) - H(Y | X)$$

Where $H(Y)$ is the entropy of the label distribution, and $H(Y | X)$ is the conditional entropy of Y given X . The conditional entropy can be computed using the weighted sum of the entropies for each value of X :

$$H(Y | X) = P(X = 0)H(Y | X = 0) + P(X = 1)H(Y | X = 1)$$

Thus, the full expression becomes:

$$I(Y; X) = H(Y) - P(X = 0)H(Y | X = 0) - P(X = 1)H(Y | X = 1)$$

It is important to correctly weight the entropy calculations by the probability of each split to ensure accurate measurement of the information gain.

This measure helps select the attribute that maximally reduces the uncertainty (entropy) about the label Y when the dataset is split based on that attribute.

3.3 Implementation

With the help of the hints part in the instruction, we can implement the program in the following steps.

3.3.1 Helper function

We need to write helper functions to calculate entropy and mutual information. Based on the definition we gave above, we can use the following code to implement the function:

```
1 def compute_entropy(Counts):
2     #Counts is the list for category. [6,4] represent 6 A 4 B
3     """compute the entropy, helper function"""
4     total = sum(Counts)
5
6     if total ==0 :
7         return 0
8
9     entropy =0.0
10
11    for i in Counts:
12        if i !=0:
13            p = i/total
14            entropy -= p*math.log2(p)
15    return entropy
```

And we can use the following code to implement the mutual information function:

```
1 def Calc_Mutual_Information(V0, V1, label_name):
2     """Use mutual information to determine which attribute to
3     ↪ split on"""
4     # calculate the num of each category
5     cnt00 = sum(1 for sample in V0 if sample[-1] ==
6     ↪ label_name[0])
7     cnt01 = sum(1 for sample in V0 if sample[-1] ==
8     ↪ label_name[1])
9     cnt10 = sum(1 for sample in V1 if sample[-1] ==
10    ↪ label_name[0])
11    cnt11 = sum(1 for sample in V1 if sample[-1] ==
12    ↪ label_name[1])
13    num0, num1 = len(V0), len(V1)
```

```
10     cnt0_total = cnt00 + cnt10
11     cnt1_total = cnt01 + cnt11
12     total = cnt0_total + cnt1_total
13
14     # compute the total entropy
15     TotEnt = compute_entropy([cnt0_total, cnt1_total])
16
17     # compute the entropy of V0
18     Ent0 = compute_entropy([cnt00, cnt01])
19
20     # compute the entropy of V1
21     Ent1 = compute_entropy([cnt10, cnt11])
22
23     # information gain
24     p0 = num0 / total if total != 0 else 0
25     p1 = num1 / total if total != 0 else 0
26     MI = TotEnt - p0 * Ent0 - p1 * Ent1
27     return MI
```

3.3.2 TreeNode class

The `TreeNode` class is a custom data structure for representing each node in the decision tree. It stores how the node splits, what it predicts (if it's a leaf), and any child nodes. This allows you to build a complete decision tree by linking `TreeNode` objects together.

In the `TreeNode` class, we define the following attributes:

- **id**: A unique identifier for the node.
- **isLeaf**: A boolean indicating whether the node is a leaf node or not.
- **crit**: A string representing the splitting criterion (e.g., Information Gain).
- **identify**: A string representing the attribute used for splitting, or the label's name.

- **children**: A list of child nodes (TreeNode objects) that represent the branches of the tree.

And we define the following methods:

- **__init__**: The constructor initializes the node with the given parameters and sets up the children list.
- **SetLeaf**: A method to set the node as a leaf node and assign a label.
- **AddNode**: A method to add a child node to the current node.

This is the key part of the decision tree implementation, as it allows you to create a hierarchical structure of nodes that can be traversed for making predictions. Here is the detailed code I used.

```
1 class TreeNode:
2     """decision tree node class"""
3
4     def __init__(self, NodeID):
5         self.id = NodeID
6         self.isLeaf = False
7         self.crit = 0
8         self.identify = None # or the label name
9         self.children = {} #children node dictionary
10        return
11
12    def SetLeaf(self, X, label_name):
13        """set the current node as leaf and classify"""
14        self.isLeaf = True
15
16        # count the num of sample
17        label_counts = Counter(sample[-1] for sample in X)
18        cnt0 = label_counts[label_name[0]]
19        cnt1 = label_counts[label_name[1]]
20
21        # majority voting method
22        if cnt0 > cnt1:
23            self.identify = label_name[0]
```

```
24         elif cnt0 < cnt1:
25             self.identify = label_name[1]
26         else:
27             self.identify = max(label_name[0], label_name[1])
28             ↪ # with the lexicographically larger value
29         return
30     def AddNode(self, Type, ChildNode):
31         """Add new child node"""
32         self.children[Type] = ChildNode
33         return
```

3.3.3 Splitting Mechanism

The `split()` function is used to recursively split the dataset X based on the informative attribute, mutual information as the criterion. It builds the decision tree one node at a time, deciding the best attribute to split on until certain stopping conditions are met.

Using the thought of recursion, the stop condition is as follows:

- **Maximum Depth:** If the current depth is equal to the maximum allowed depth, stop and set the node as a leaf
- **All Labels are the Same:** If all samples in X have the same label, the function stops and sets the node as a leaf.
- **No Available Splits:** If no attribute can provide a meaningful split (i.e., mutual information is 0 or negative), the node is set as a leaf.

The main logic is as follows:

- **Calculate Mutual Information:** For each attribute (that has not been used for splitting yet), the function calculates the mutual information (MI) between the two resulting subsets V_0 and V_1 (created by splitting on that attribute).

- Choose the Best Attribute: The attribute that maximizes the mutual information is selected to split the data. If there is no attribute that provides a meaningful split ($MI \leq 0$), the current node is set as a leaf.
- Recursively Split: If a valid split is found, the function creates two child nodes, one for each possible value of the selected attribute. It then recursively splits the subsets V_0 and V_1 created by the split, calling the `split()` function again.
- Leaf Node Creation: When the recursive calls reach a stopping condition, the current node is set as a leaf node with the most frequent label in the subset X .

Here is the detailed code I used:

```
1 def split(X, nowNode, now_depth, max_depth, isSplited,  
2   ↪ attr_name, label_name):  
3     """split the data to construct decision tree"""  
4     # recursion terminate condition  
5     # 1 reach the max depth  
6     if now_depth == max_depth:  
7         nowNode.SetLeaf(X, label_name)  
8         return  
9  
10    # 2 all the categories are same.  
11    if all(sample[-1] == X[0][-1] for sample in X):  
12        nowNode.SetLeaf(X, label_name)  
13        return  
14    if not X :  
15        return  
16  
17    max_ent_ind, max_MI = -1, 0  
18    for i, tag in enumerate(isSplited):  
19        if tag == 1:  
20            continue  
21        # split
```

```

21     V0 = [sample for sample in X if sample[i] ==
           ↪ attr_name[i][0]]
22     V1 = [sample for sample in X if sample[i] ==
           ↪ attr_name[i][1]]
23     if len(V0) == 0 or len(V1) == 0:
24         continue
25
26     MI = Calc_Mutual_Information(V0, V1, label_name) #
           ↪ Use mutual information to determine which
           ↪ attribute to split on
27
28     if MI > max_MI:
29         max_MI = MI
30         max_ent_ind = i
31
32     # 3 there is no attribute to use.
33     if max_ent_ind == -1 or MI <= 0:
34         # only split on an attribute if the mutual
           ↪ information is > 0
35         nowNode.SetLeaf(X, label_name)
36         return
37
38     # create and split
39     global NodeCnt
40     nowNode.crit = max_ent_ind
41     isSplited[max_ent_ind] = 1
42
43     # attribute 1
44     NodeCnt += 1
45     newNode = TreeNode(NodeCnt)
46     nowNode.AddNode(attr_name[max_ent_ind][0], newNode)
47     V0 = [sample for sample in X if sample[max_ent_ind] ==
           ↪ attr_name[max_ent_ind][0]]
48     split(V0, newNode, now_depth + 1, max_depth, isSplited.
           ↪ copy(), attr_name, label_name)
49
50     # attribute 2
51     NodeCnt += 1

```

```
52     newNode = TreeNode(NodeCnt)
53     nowNode.AddNode(attr_name[max_ent_ind][1], newNode)
54     V1 = [sample for sample in X if sample[max_ent_ind] ==
           ↪ attr_name[max_ent_ind][1]]
55     split(V1, newNode, now_depth + 1, max_depth, isSplited.
           ↪ copy(), attr_name, label_name)
56
57     return
```

3.3.4 Build Decision Tree

Then we can use the functions above to build the decision tree.

```
1 def Gen_Tree(X, max_depth, attr_name, label_name, attr_title)
   ↪ :
2     """Generate the decision tree """
3     global NodeCnt
4     NodeCnt += 1
5     root = TreeNode(NodeCnt) # create the root node
6     isSplited = [0] * len(attr_name) # initialize the
           ↪ isSplited list
7     split(X, root, 0, max_depth, isSplited, attr_name,
           ↪ label_name)
8     return root
```

3.3.5 Task function

To finish the given tasks, we need Prediction and Print functions. The Prediction function is used to predict the labels of the training and testing examples, and the Print function is used to print the decision tree in a specified format.

Here is the code I used to implement the Prediction function:

```
1 def Prediction(root, X, out_file):
2     """prediction """
3     pred_label = []
4     for data in X:
5         nowNode = root
```

```
6
7     while not nowNode.isLeaf:
8         attr_val = data[nowNode.crit]
9         nowNode = nowNode.children[attr_val]
10        pred_label.append(nowNode.identify)
11
12    with open(out_file, "w") as f:
13        for label in pred_label:
14            f.write(f"{label}\n")
15
16    error = sum(1 for i in range(len(pred_label)) if
17               ↪ pred_label[i] != X[i][-1]) / len(X)
18    return error
```

And here is the `code` I used to implement the Print function:

```
1 def printTree(nowNode, X, dep, attr_name, label_name,
2   ↪ attr_title):
3
4     if dep == 0:
5         cnt0 = sum(1 for sample in X if sample[-1] ==
6   ↪ label_name[0])
7         cnt1 = len(X) - cnt0
8         print(f"[{cnt0} {label_name[0]}/{cnt1} {label_name
9   ↪ [1]}]")
10
11    if nowNode.isLeaf:
12        return
13
14    feature_id = nowNode.crit
15    V0 = [sample for sample in X if sample[feature_id] ==
16   ↪ attr_name[feature_id][0]]
17    V1 = [sample for sample in X if sample[feature_id] ==
18   ↪ attr_name[feature_id][1]]
19
20    cnt0 = sum(1 for sample in V0 if sample[-1] == label_name
21   ↪ [0])
22    cnt1 = len(V0) - cnt0
23    print(" / " * (
```

```

18         dep + 1) + f"{attr_title[feature_id]} = {
           ↪ attr_name[feature_id][0]}: [{cnt0} {
           ↪ label_name[0]}/{cnt1} {label_name[1]}} "
           ↪ )
19     printTree(nowNode.children[attr_name[feature_id][0]], V0,
           ↪ dep + 1, attr_name, label_name, attr_title)
20
21     cnt0 = sum(1 for sample in V1 if sample[-1] == label_name
           ↪ [0])
22     cnt1 = len(V1) - cnt0
23     print("/ " * (
24         dep + 1) + f"{attr_title[feature_id]} = {
           ↪ attr_name[feature_id][1]}: [{cnt0} {
           ↪ label_name[0]}/{cnt1} {label_name[1]}} "
           ↪ )
25     printTree(nowNode.children[attr_name[feature_id][1]], V1,
           ↪ dep + 1, attr_name, label_name, attr_title)

```

In this way, we can print the decision tree in the command line and also it is in the given format.

3.3.6 Program Process

To achieve the goal of the program, we also need the Read_Data function to read the data from the file.

```

1 def Read_Data(trainfile, testfile):
2     """Read TSV training and testing files and extract
           ↪ attributes and labels"""
3     InputData_train = pd.read_csv(trainfile, delimiter='\t')
4     InputData_test = pd.read_csv(testfile, delimiter='\t')
5     # to the str list
6     train_input = InputData_train.astype(str).values.tolist()
7     test_input = InputData_test.astype(str).values.tolist()
8     attr_name = []
9     for col in InputData_train.columns[:-1]: # 不包括标签列
10         unique_vals = sorted(InputData_train[col].astype(str)
           ↪ .unique().tolist())
11         if len(unique_vals) < 2:

```

```

12         unique_vals.append("null")
13         attr_name.append(unique_vals)
14     label_name = sorted(InputData_train.iloc[:, -1].astype(
15         ↪ str).unique().tolist())
16     attr_title = InputData_train.columns[:-1].astype(str).
17         ↪ tolist()
18     return train_input, test_input, attr_name, label_name,
19         ↪ attr_title

```

Finally, we can combine these functions into the main function. Here is the process of the program.

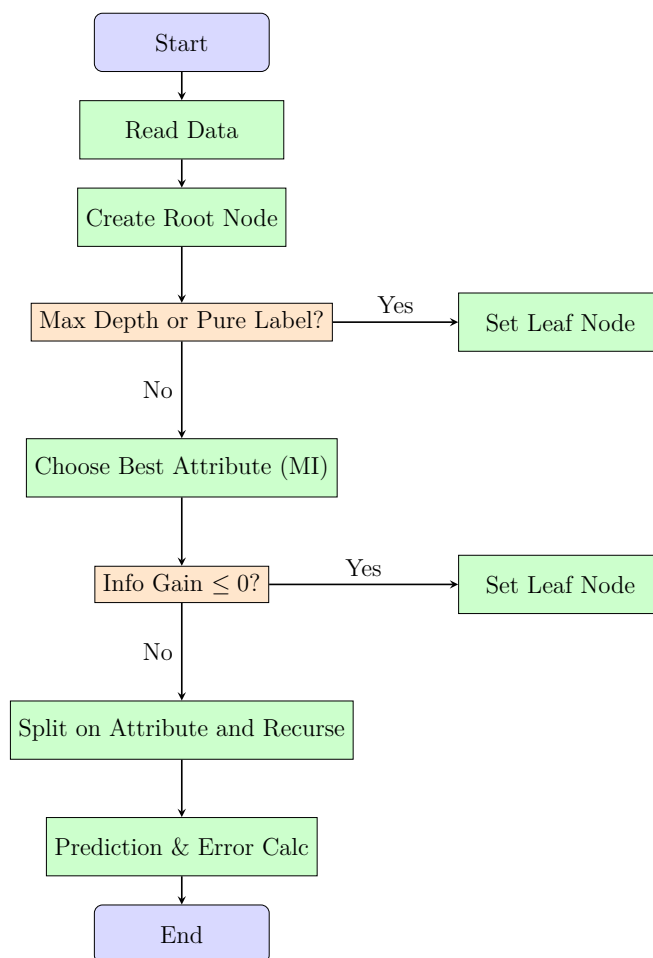


Figure 2: Decision Tree Program Flowchart

Refer to the flowchart above, we can implement the main function as follows:

```
1 if __name__ == "__main__":
2     # invalid input
3     if len(sys.argv) < 7:
4         print(
5             "Usage: python3 decisionTree.py <train input> <
              ↳ test input> <max depth> <train out> <test
              ↳ out> <metrics out>")
6         sys.exit(1)
7
8     _, train_input, test_input, max_depth, train_out,
9     ↳ test_out, metrics_out = sys.argv
10    max_depth = int(max_depth)
11
12    # read
13    train_data, test_data, attr_info, labels, attr_titles =
14    ↳ Read_Data(train_input, test_input)
15
16    # construct
17    decision_tree = Gen_Tree(train_data, max_depth, attr_info
18    ↳ , labels, attr_titles)
19
20    # predict and save
21    train_error = Prediction(decision_tree, train_data,
22    ↳ train_out)
23    test_error = Prediction(decision_tree, test_data,
24    ↳ test_out)
25
26    # output
27    with open(metrics_out, "w") as f:
28        f.write(f"error(train): {train_error}\nerror(test): {
29        ↳ test_error}")
30
31    # print out the struct
32    printTree(decision_tree, train_data, 0, attr_info, labels
33    ↳ , attr_titles)
```

3.4 Test

After we implement the program, we need to test it. We use the following command to run the program.

```
1 python3 decisionTree.py <train input> <test input> <max depth  
  → > <train out> <test out> <metrics out>
```

3.4.1 Test 1

For file small and max depth 3, we can get the results as follows: 1. Decision Tree structure is as shown:

```
[15 democrat/13 republican]  
| Anti_satellite_test_ban = n: [2 democrat/12 republican]  
| | Export_south_africa = n: [0 democrat/5 republican]  
| | Export_south_africa = y: [2 democrat/7 republican]  
| Anti_satellite_test_ban = y: [13 democrat/1 republican]  
| | Export_south_africa = n: [0 democrat/1 republican]  
| | Export_south_africa = y: [13 democrat/0 republican]
```

Figure 3: The struct of Decision Tree

which is same as the given result. 2. The error rate is as follows:

```
error(train): 0.07142857142857142  
error(test): 0.14285714285714285
```

Figure 4: Error rate

which is same as the contents in handout. 3. The labels of Prediction are as follows:

1	republican	15	republican
2	republican	16	republican
3	democrat	17	democrat
4	democrat	18	democrat
5	democrat	19	democrat
6	democrat	20	democrat
7	democrat	21	republican
8	republican	22	democrat
9	democrat	23	republican
10	republican	24	republican
11	democrat	25	republican
12	republican	26	republican
13	democrat	27	republican
14	republican	28	republican

Figure 5: Prediction Results

which is same as the contents in handout.

3.4.2 Test 2

For file small and max depth 2, we can get the results as follows: 1. Decision Tree structure is as shown:

```
[15 democrat/13 republican]
| Anti_satellite_test_ban = n: [2 democrat/12 republican]
| | Export_south_africa = n: [0 democrat/5 republican]
| | Export_south_africa = y: [2 democrat/7 republican]
| Anti_satellite_test_ban = y: [13 democrat/1 republican]
| | Export_south_africa = n: [0 democrat/1 republican]
| | Export_south_africa = y: [13 democrat/0 republican]
```

Figure 6: The struct of Decision Tree

which is same as the given result. 2. The error rate is as follows:

```
1 error(train): 0.07142857142857142
2 error(test): 0.14285714285714285
```

Figure 7: Error rate

which is same as the contents in handout. 3. The labels of Prediction are as follows:

1	republican	15	democrat
2	republican	16	republican
3	republican	17	republican
4	republican	18	democrat
5	republican	19	democrat
6	democrat	20	republican
7	republican	21	republican
8	republican	22	republican
9	democrat	23	republican
10	republican	24	republican
11	democrat	25	republican
12	republican	26	republican
13	republican	27	democrat
14	democrat	28	democrat

Figure 8: Prediction Results

which is same as the contents in handout.

After we test the program, we find it matches the given results, which prove our programs are rightt.

3.5 Perfomance Analysis

Then we run the program on the 4 datasets. Because, the variable, `max_depth`, is the main parameter to influence the prediction results, we can set it to 4 for all the datasets.

3.5.1 Relation between `max_depth` and error rate

We can get the error rate as follows:

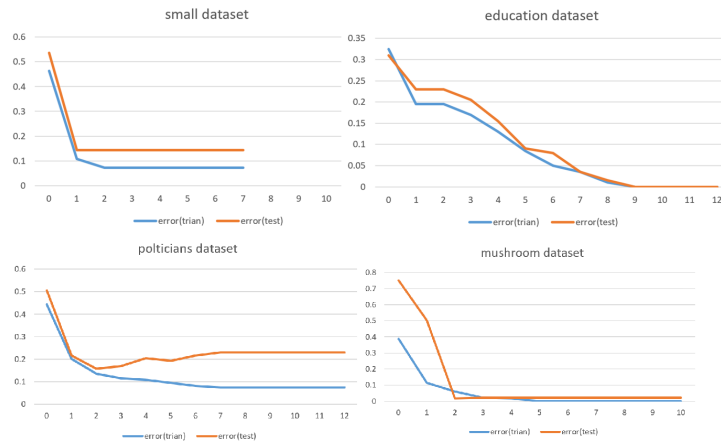


Figure 9: The relation between `max_depth` and error rate

Training Error: As the depth of the tree increases, the training error gradually decreases and approaches zero. This is because deeper trees can capture more details in the training data, allowing the model to fit the data better. As the depth of the tree increases, the model becomes more complex, eventually overfitting the training data, causing the training error to approach zero.

Testing Error: The testing error exhibits a more complex trend. Initially (with shallower tree depths), the testing error is high because the model cannot capture the complex patterns in the data. However, as the depth of the tree increases, the testing error decreases until it starts to rise beyond a certain point. This is because as the depth increases, the model begins to overfit the training data, causing it to generalize poorly on the test set, leading to an increase in the testing error.

However, we found that not all the dataset final error rate is 0.0. For politicians dataset, the error rate converge to 0.0738255033557047.

Here is the possible explanation:

- As the tree depth increases, the training error approaches zero, but the testing error begins to rise beyond a certain depth, indicating the occurrence of overfitting.

- The generalization ability of a decision tree may vary across different datasets. Some datasets are more adaptable to the model, meaning that even with increased depth, overfitting does not occur easily, and the error eventually converges to a relatively low value. For other datasets, it may be difficult for the model to adapt without overfitting, so even as the depth increases, the error still persists. That's may the reason why the error in politicians didn't converge to 0.0.
- Choosing the appropriate tree depth is crucial for optimizing the model. A tree that is too deep may overfit, while a shallow tree may not capture enough information. Using techniques like cross-validation can help select the best tree depth.

3.5.2 Futher Work

To deal with the overfitting problem, we can use the following methods:

- Pruning: This involves removing branches from the tree that have little importance, which can help reduce overfitting.
- Cross-validation: This technique can be used to select the optimal tree depth by evaluating the model's performance on different subsets of the data.
- Ensemble methods: Techniques like bagging and boosting can be used to combine multiple decision trees, which can improve generalization and reduce overfitting.

4 Disucuss and Conclusion

4.1 Python3? Python? Python!

When run the inspection.py in the command line, I use "python3 inspection.py <input> <output>" this command, and it worked. However, when

I use "python3 decisionTree.py <train input> <test input> <max depth> <train out> <test out> <metrics out>" to run decisionTree.py, it didn't work. Then I try to use python instead of python3, and it worked. Some of my classmates also met this problem.

Till now, I have no idea why this happened. I think it may be the version of python3 is not compatible with the code I wrote? It also remains me that when I work on the project in the CS61A, I also met this problem. Maybe it is the same reason?

But in all, it trains me to use the command line, maybe can help me learn linux better in the furtuer. Thank you for the instruction.

4.2 Conclusion

This machine learning assignment focuses on implementing a decision tree classifier using mutual information as the splitting criterion. The key tasks include building a decision tree from training data, applying pruning based on a specified maximum depth, predicting outcomes on the test set, and reporting both training and testing errors. The implementation requires handling command-line arguments, processing tsv files, generating prediction outputs, and printing the structure of the decision tree to the console.

Through this assignment, I gained a deeper understanding of how decision trees work, particularly how mutual information guides feature selection during the split. Moreover, the emphasis on modular design and file-based input/output enhanced my ability to write reusable and well-structured code. This assignment significantly improved both my algorithmic thinking and practical programming skills.

5 AI Using Report

Of crouse, I use AI to finish the experiments and report. I also highlight the part I use AI in this report. Here is the Conclusion.

1. Actually, this is my first time to use LaTeX to write a report. So, I use AI a lot to write the code and debug. (Till now, I still think Latex is hard to use. In the future, I may change my minds.)
2. tsv file, I have never met it before(...forgive me, plz teacher.). So I use ChatGPT to help me introduce the read or write operations.(GPT-4o gives some free try everyday, but not enough,orz)
3. To be honest, I didn't write the code from 0, I use the code in Github and ChatGPT to help me write the code. But plz trust me, I didn't ctrl c+ctrl v all the time! And also, the codes are all from my own thoughts!
4. Finally, I use Deepseek to help me with the English grammar and make the report more standardized.

References

- [1] 周志华. 《机器学习》. 清华大学出版社, 2016.
- [2] Andrew Ng. "Coursera-ML-AndrewNg-Notes" <https://github.com/fengdu78/Coursera-ML-AndrewNg-Notes>